

Tower of Hanoi 3D - Project Documentation

Overview

This project is a **3D physics-based Tower of Hanoi puzzle game** built in Unity (version 6000.2.2f1) using C# and the MVC (Model-View-Controller) design pattern.

Players can drag and drop disks between three towers, choose between 3-10 disks, and track their move count. The game features smooth elastic physics, visual feedback, and options for ordered or random disk spawning.

The game validates moves according to traditional Hanoi rules:

1. Only one disk can be moved at a time.
2. A larger disk cannot be placed on top of a smaller disk.
3. The goal is to move all disks from Tower A to Tower C.

Features

Core Gameplay

- **Physics-Based Interaction:** Disks use elastic spring physics for smooth mouse following.
- **Visual Feedback:** Outline effects (green = valid, red = invalid) on hover.
- **Audio Feedback:** Sounds for pickup, release, invalid moves, and victory.
- **Move Counter:** Tracks the number of moves made.
- **Victory Detection:** Automatic win condition checking.

Customization

- **Configurable Disk Count:** Select 3-10 disks via an interactive cylinder selector.
- **Spawn Order Toggle:** Choose ordered (largest to smallest) or random disk spawning.
- **Reset Functionality:** A reset button to restart the game with current settings.

User Interface

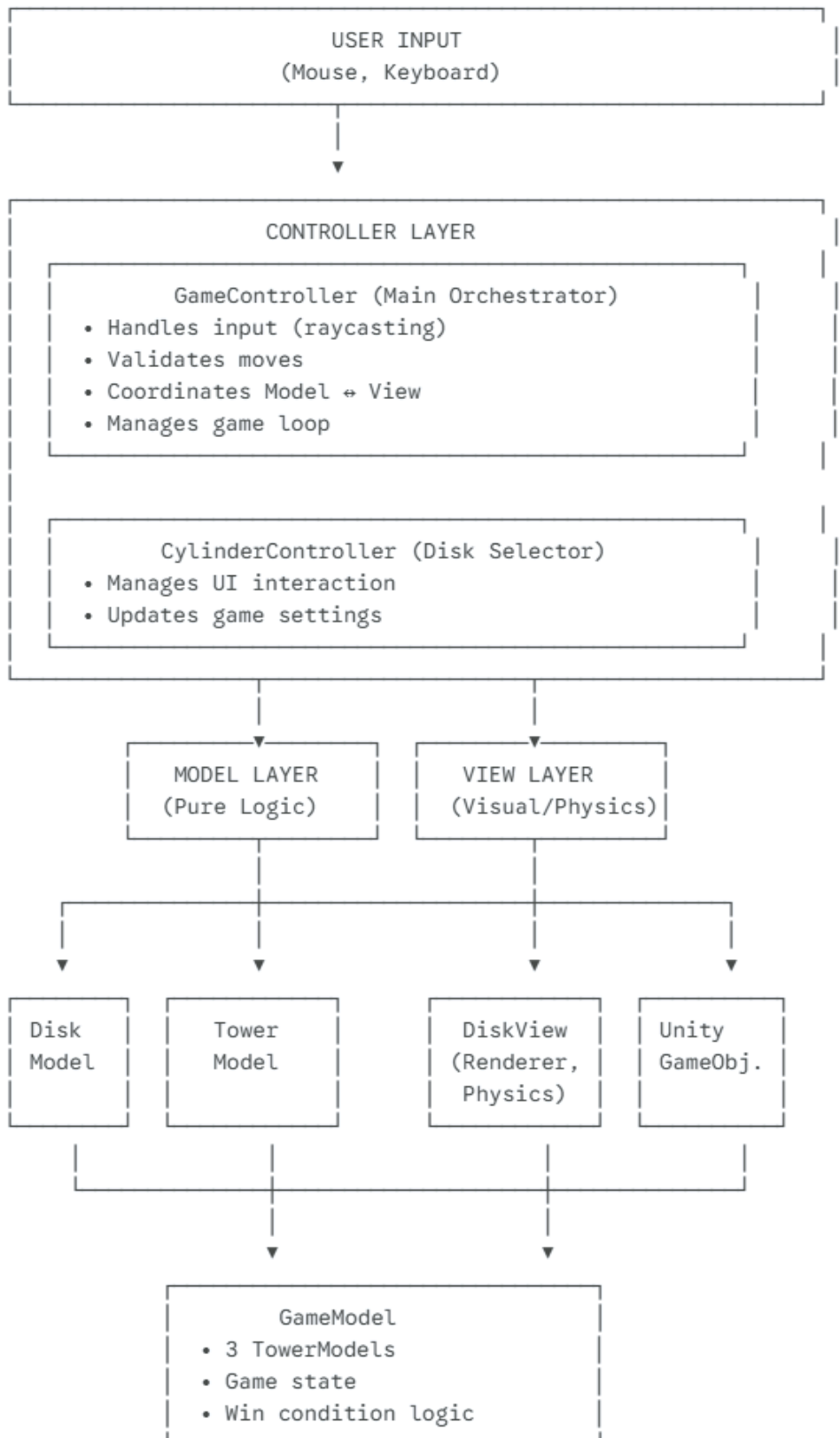
- **Reset Button:** Restarts the game.
- **Order Button:** Toggles spawn order (LED indicator: green = ordered, red = random).
- **Quit Button:** Exits the application.
- **Move Counter:** Real-time display of moves made.



Technical Architecture

MVC Design Pattern

The project follows a strict **Model-View-Controller** pattern to separate concerns.



Layer Breakdown

1. Model Layer (Data & Logic)

Purpose: Pure game logic and data structures with no Unity dependencies (except `Debug` for logging). This makes the core logic independently testable.

- **DiskModel.cs:**
 - **Properties:** `Size` (int), `TowerIndex` (int).
 - **Design:** Immutable `Size` (set only in constructor) prevents accidental modification.
- **TowerModel.cs:**
 - **Data Structure:** Uses `Stack<DiskModel>` to perfectly mirror the LIFO (Last In, First Out) nature of a physical tower.
 - **Methods:** `Push()`, `Pop()`, `Peek()`, `Clear()`.
- **GameModel.cs:**
 - **Purpose:** The "source of truth" for the entire game state.
 - **Properties:** `DiskCount`, `MoveCount`, `Towers[3]` (an array of `TowerModel`).
 - **Methods:** `Initialize()`, `MoveDisk()`, `IsGameComplete()`.

2. View Layer (Visual Representation)

Purpose: Bridges the gap between the logical data (Models) and the visual GameObjects in the Unity scene.

- **DiskView.cs:**
 - **Responsibilities:** Handles rendering, physics, user interaction, and visual feedback for a single disk.
 - **Physics:** Implements the elastic spring dragging system.
 - **Feedback:** Manages outline materials (green/red) on hover.
 - **Validation:** Contains logic for checking landing positions *after* physics settle.

3. Controller Layer (Orchestration)

Purpose: The "brain" of the game. It handles user input, communicates with the Model to update state, and tells the View to reflect those changes.

- **GameController.cs:**
 - **Responsibilities:** Central manager coordinating all systems.
 - **Input:** Handles mouse input via raycasting to detect disk selection.
 - **Validation:** Validates moves against Hanoi rules by consulting the `GameModel`.
 - **Coordination:** Updates the `GameModel` first, then instructs the `DiskView` to move. Manages UI updates and victory checks.
- **CylinderController.cs:**
 - **Purpose:** Manages the disk count selector UI.
 - **Function:** Handles dragging, snapping to the nearest value, and updating the `GameController` with the new `diskCount`.



Key Technical Features & Design Decisions

1. Physics-Based Interaction

Instead of rigidly snapping the disk to the cursor, an elastic spring system is used.

```
Force = (targetPosition - currentPosition) * springStrength - currentVelocity * springDamping
```

- **Result:** This creates a smooth, organic, and tactile movement that feels natural, as if the disk has weight and momentum.
- **Why:** Provides a superior User Experience (UX) compared to simple cursor-following or Lerp-based movement.

2. Smart Move Validation

Move validation happens *after* the user releases the disk, not during the drag.

- **Flow:** Player drags disk → Physics settles (0.3s delay) → Validate position and rules.
- **Why:**
 - **Reliability:** Physics needs time to settle to get an accurate final position.
 - **UX:** Players expect an immediate drag response, not instant rejection. It feels more natural to check the move on release.

3. Visual Feedback System

The game provides immediate, non-intrusive feedback.

- **Hovering:**
 - **Green Outline:** The disk is on top of a tower and can be picked up.
 - **Red Outline:** The disk is not on top and cannot be moved.
- **Why:** This clearly communicates game rules to the player without text, teaching them the "only move the top disk" rule organically.

4. Design Pattern: Why MVC?

- **Testability:** The Model layer (game logic, rules) can be unit-tested entirely separately from Unity's visual engine.
- **Scalability:** New features are easy to add. An "Undo" system, for example, only needs to modify the GameModel and GameController, leaving the View layer untouched.
- **Maintainability:** Clear boundaries prevent "spaghetti code." Logic, data, and visuals are all in their own distinct, predictable places.



Technical Challenges Solved

- **Challenge 1: Disk Stacking**
 - **Problem:** Disks need to stack perfectly aligned on top of each other.
 - **Solution:** The target stack position is calculated based on the `TowerModel.Count` and a fixed disk height, ensuring perfect alignment.
- **Challenge 2: Rule Validation**
 - **Problem:** Prevent invalid moves (e.g., placing a large disk on a small one or placing a disk out of a tower).
 - **Solution:** All move validation checks the `GameModel` first. The visual move only happens *after* the logical move is confirmed valid.
- **Challenge 3: Smooth Dragging**
 - **Problem:** Rigid cursor-following feels robotic and unnatural.
 - **Solution:** Implemented the elastic spring physics system with configurable strength and damping.
- **Challenge 4: Reset Without Bugs**
 - **Problem:** Resetting a disk's position to its "previous" spot could be buggy if it had moved.
 - **Solution:** An invalid move resets the disk to its *current* logical tower position (based on `diskModel.TowerIndex`), not a stored initial position.



Performance Optimizations

1. **Component Caching:** `Camera.main`, `Rigidbody`, and `Renderer` references are cached in `Awake()` to avoid repeated `GetComponent()` calls.
2. **Efficient Queries:** Uses `FindObjectsByType(FindObjectsSortMode.None)` for optimal performance when finding objects.
3. **Physics Constraints:** Disk `Rigidbody` components have rotation frozen to prevent unwanted tumbling.
4. **Smart Raycasting:** Only a single raycast is performed per frame in `Update()` for mouse input, not multiple.



Project Structure

```
Hanoi3D/
├── Assets/
│   ├── Scripts/
│   │   ├── Models/                # Data models (MVC Model layer)
│   │   │   ├── DiskModel.cs
│   │   │   ├── TowerModel.cs
│   │   │   └── GameModel.cs
│   │   ├── Views/                # Visual representations (MVC View layer)
│   │   │   └── DiskView.cs
│   │   ├── Controllers/          # Game logic controllers (MVC Controller layer)
│   │   │   ├── GameController.cs
│   │   │   └── CylinderController.cs
│   │   └── UI/                   # UI interaction scripts
│   │       ├── ResetButton.cs
│   │       ├── OrderButton.cs
│   │       └── QuitButton.cs
│   ├── Prefabs/
│   │   └── DiskPrefab             # Disk GameObject with DiskView component
│   ├── Materials/
│   ├── Audio/
│   └── ... (Other Unity assets)
└── README.md
```



Scripts Documentation

Model Scripts

- **DiskModel.cs**
 - A pure C# data class representing a single disk.
 - **Properties:** `Size` (int), `TowerIndex` (int).
 - **Purpose:** Stores the logical state of a disk without game logic.
- **TowerModel.cs**
 - A data structure class that uses a `Stack<DiskModel>`.
 - **Methods:** `Push()`, `Pop()`, `Peek()`, `Clear()`.
 - **Purpose:** Manages the logical stack of disks on one tower.
- **GameModel.cs**
 - The central game state manager; the "source of truth."
 - **Properties:** `DiskCount`, `MoveCount`, `Towers[]` (an array of `TowerModel`).
 - **Methods:** `Initialize()`, `MoveDisk()`, `IsGameComplete()`.

View Scripts

- **DiskView.cs**
 - The visual representation of a disk (a `MonoBehaviour`).
 - **Physics:** Manages the `Rigidbody` and implements elastic spring dragging.
 - **Visuals:** Handles outline materials (green/red) on hover.
 - **Validation:** Checks landing position and communicates with `GameController` to validate moves.

Controller Scripts

- **GameController.cs**
 - The main game orchestrator (a `MonoBehaviour`).
 - **Initialization:** Spawns disks and sets up the `GameModel`.
 - **Input:** Handles mouse raycasting for disk selection.
 - **Validation:** Enforces Hanoi rules by checking the `GameModel`.
 - **UI Management:** Updates the move counter text and victory screen.
- **CylinderController.cs**
 - Manages the disk count selector UI.
 - **Interaction:** Handles drag-and-drop logic along the X-axis.
 - **Snapping:** Snaps to the nearest valid disk count.

UI Scripts

- **ResetButton.cs:** Calls `GameController.ResetGameWithDiskCount()` and plays a press animation.
- **OrderButton.cs:** Toggles the `GameController.spawnDisksOrdered` boolean and updates the LED material.
- **QuitButton.cs:** Calls `Application.Quit()` (functions in builds only).

✓ Code Quality Practices

- **XML Documentation:** All public methods and properties are documented with C# XML comments.
- **Regions:** Code within scripts is organized into logical `#region` blocks (e.g., Initialization, Public Methods, Private Methods, UI).
- **Naming Conventions:** Follows standard C# and Unity naming conventions.
- **Error Handling:** Defensive checks are in place (e.g., null checks) to prevent common errors.
- **No Magic Numbers:** Key values (e.g., spring strength, move delay) are exposed as `[SerializeField]` variables rather than hard-coded.

✖ Potential Future Improvements

- **Hint System:** Suggest the next valid move.
- **Timer Mode:** Challenge mode with time limits.
- **Move History:** A UI list of all moves with undo/redo capability.
- **Object Pooling:** Use object pooling for disk instantiation/destruction on reset for better performance, especially with high disk counts.
- **AR/VR/Mobile Support:** Adapt controls and UI for other platforms.

⚠ Known Limitations

1. **Single Scene Only:** The game uses a single scene; there is no main menu or level system, this was a design choice.
2. **No Save System:** The game state is not persisted between sessions.
3. **No Undo Feature:** A move-reversal feature is not currently implemented.
4. **Desktop Only:** The game is not optimized for mobile/touch input.



System Requirements

- **Unity Version:** 6000.2.2f1 or newer
- **Platform:** Windows, macOS, or Linux
- **Recommended Hardware:** Mid-range PC with a dedicated GPU

⚠ Final Considerations on AI Usage

I used the Unity 6.0 AI to help me with some very long methods.

Initially, I had started doing everything by hand, but then, realizing it would take too much time to deliver the project in under 10 days, I decided to work with "vibe coding."

Essentially, I told the AI what to do and how to implement the methods, their logic, and their data structures, such as the stack for the set of disks.

Most of the scripts were generated by the AI but then manually reviewed and fixed by me. I encountered several bugs; perhaps the hardest and most complicated part to implement was resetting a disk's position. I spent an entire day figuring out how to precisely get the disk's last position before it was moved, and I had to write the `ResetGameWithDiskCount()` method by hand.

I know you probably won't be happy to hear that the scripts were made with AI, but I prefer to be honest with you. In 2025, not using AI is like being an accountant and not using a calculator.

This documentation was initially written by hand and then revised by Claude AI. The same goes for the comments; I initially put them in by hand in English and asked Claude for help to translate them correctly and indent them well.